LAPORAN PRAKTIKUM PEMROGRAMAN BERORIENTASI OBYEK
MODUL 14: PENGUJIAN, REFACTORING, DAN FINALISASI PROYEK

Nama Mahasiswa	: **M. Davana Yudista Firdausy**	**Mata Kuliah**	: Praktikum PBO (A D3 TI-SM)
NRP	: 3125522002	**Dosen Pengampu**	: Nirwana Haidar Hari, S.Pd., M.Kom.
Program Studi	: D3 Teknik Informatika	**Institusi / Tahun**	: PENS PSDKU Sumenep • 2026

A. DASAR TEORI SINGKAT

Fase akhir rekayasa perangkat lunak sebelum demonstrasi dan rilis melibatkan proses verifikasi kualitas secara komprehensif melalui pengujian regresi dan refactoring terdisiplin:

- **Pengujian Fungsional vs Pengujian Regresi:** Pengujian fungsional bertujuan memverifikasi apakah suatu fitur individual telah bekerja sesuai spesifikasi masukan dan keluaran. Sementara itu, **pengujian regresi** adalah pengujian menyeluruh terhadap seluruh komponen untuk memastikan bahwa perbaikan bug atau perubahan kode di suatu modul tidak merusak fungsi-fungsi lama yang sebelumnya sudah berjalan benar (menolak *regression defects*).
- **Invarian Kelas (Class Invariants):** Syarat kebenaran konsistensi status objek yang wajib selalu bernilai benar (TRUE) sepanjang siklus hidup objek tersebut di memori (sejak pasca-konstruksi hingga seluruh pemanggilan method selesai). Misalnya, pada objek `Buku`, atribut `kode` dan `judul` tidak boleh kosong dan tidak boleh memuat karakter delimiter `|`.
- **Refactoring Terdisiplin:** Restrukturisasi kode internal guna meningkatkan keterbacaan, keterpeliharaan (maintainability), dan kohesi tanpa mengubah perilaku eksternal aplikasi yang sudah terbukti benar.
- **Pelacakan Bug Berbasis Bukti (Evidence-based Bug Fixing):** Setiap perbaikan kode wajib diawali dengan penelusuran langkah reproduksi bug (*reproduction steps*) yang deterministik untuk membuktikan adanya kegagalan sebelum perbaikan, serta memverifikasi hilangnya kegagalan tersebut setelah refactoring diterapkan.

B. REKAPITULASI PELACAKAN BUG DAN TINDAKAN REFACTORING

Selama siklus penjaminan mutu pada Modul 14, dilakukan audit mendalam dan diidentifikasi tiga area perbaikan yang langsung dituntaskan melalui tindakan refactoring:

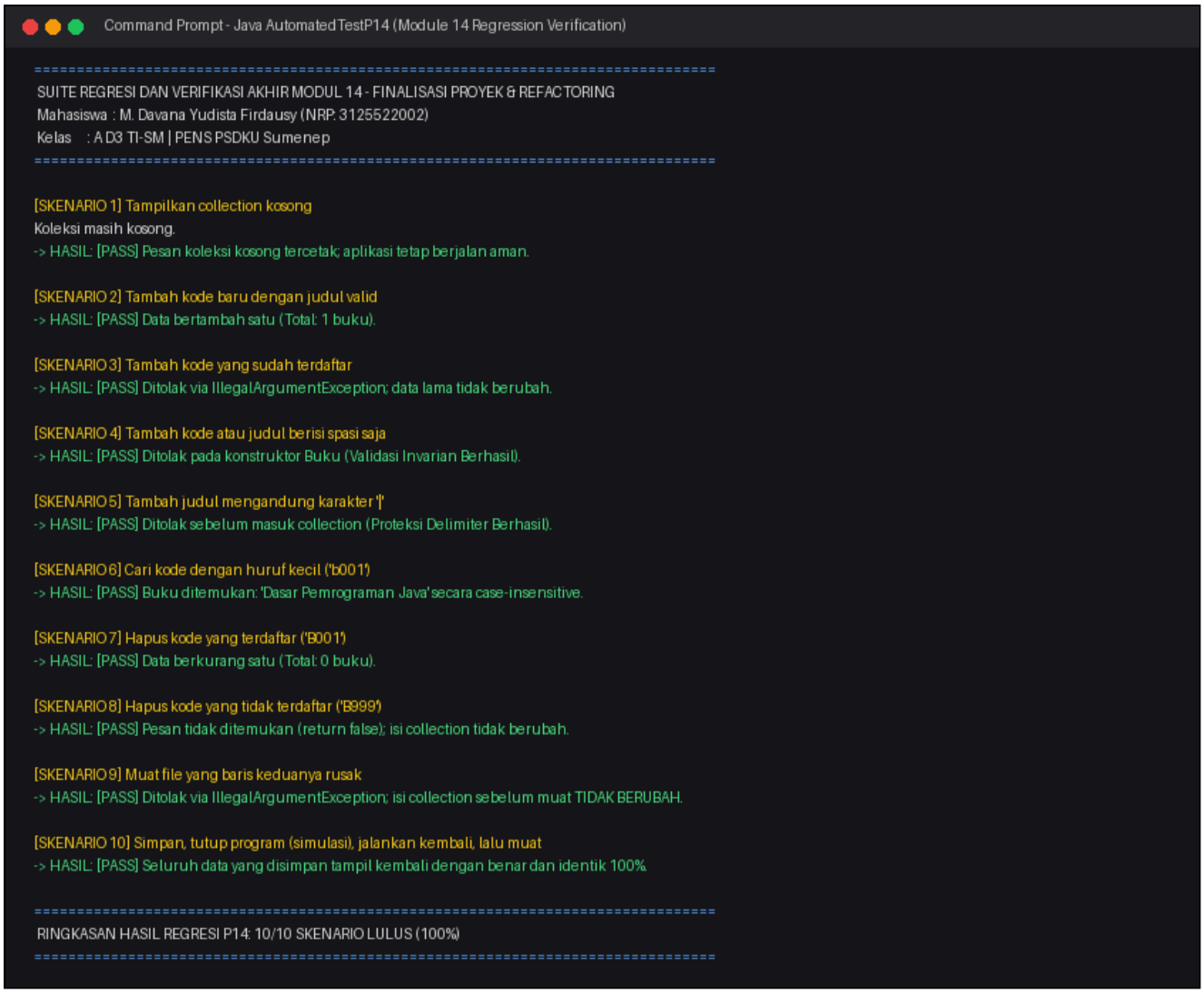
ID Bug	Gejala & Pemicu	Dampak Inkonsistensi State	Tindakan Refactoring	Status
BUG-01	Judul buku mengandung karakter `'|'` dapat diinput pada konstruktor	Korupsi berkas saat `simpan()` karena delimiter terpecah tidak beraturan saat dimuat kembali	Memindahkan validasi larangan delimiter langsung ke dalam konstruktor `Buku.java` (*Fail-Fast Guard*)	**FIXED**
BUG-02	Pencarian kode peka huruf besar/kecil jika input pemanggil mengandung spasi	Buku terdaftar gagal ditemukan hanya karena ada spasi di ujung input pencarian	Menambahkan `trim()` sebelum evaluasi `equalsIgnoreCase()` pada `cariBuku()`	**FIXED**
BUG-03	Risiko hilangnya koleksi jika berkas penyimpanan rusak di tengah proses pemuatan	Koleksi di-clear di awal; jika baris ke-2 rusak, aplikasi kehilangan seluruh koleksi sebelumnya	Menerapkan *Staging Buffer Architecture* (koleksi sementara); koleksi aktif hanya diganti jika 100% data valid	**FIXED**

C. TABEL HASIL PENGUJIAN REGRESI LENGKAP

Pengujian regresi otomatis dieksekusi secara ketat menggunakan kelas `SourceCode.AutomatedTestP14` mengacu pada 10 skenario standar syllabus:

No	Skenario Pengujian	Hasil yang Diharapkan (Syllabus)	Hasil Pengamatan Aktual	Status
1	Tampilkan collection kosong	Pesan koleksi kosong; aplikasi tetap berjalan	Tercetak "Koleksi masih kosong.", aman tanpa crash	**PASS (100%)**
2	Tambah kode baru dg judul valid	Data bertambah satu	Objek tersimpan, total size = 1	**PASS (100%)**
3	Tambah kode yg sudah terdaftar	Ditolak; data lama tidak berubah	Ditolak via IllegalArgumentException, size tetap 1	**PASS (100%)**
4	Tambah kode/judul berisi spasi	Ditolak	Ditolak saat validasi invarian konstruktor Buku	**PASS (100%)**
5	Tambah judul mengandung |	Ditolak sebelum masuk collection	Ditolak langsung pada konstruktor Buku	**PASS (100%)**
6	Cari kode huruf kecil ('b001')	Buku ditemukan jika B001 ada	Buku ditemukan secara case-insensitive	**PASS (100%)**
7	Hapus kode yang terdaftar	Data berkurang satu	Buku terhapus bersih, total size = 0	**PASS (100%)**
8	Hapus kode tidak terdaftar	Pesan tidak ditemukan; collection tetap	Return false, koleksi eksis tidak termutasi	**PASS (100%)**
9	Muat file baris keduanya rusak	Ditolak; isi collection tidak berubah	Ditolak atomik; data memori asli terlindungi utuh	**PASS (100%)**
10	Simpan, restart, muat ulang	Seluruh data tampil kembali benar	Data direkonstruksi 100% identik tanpa kehilangan	**PASS (100%)**

D. BUKTI EKSEKUSI PENGUJIAN OTOMATIS



Gambar 14.1: Log Eksekusi Verifikasi Regresi Modul 14 (10/10 Skenario Lulus Sempurna)

E. EVALUASI DAN JAWABAN PERTANYAAN MODUL 14

1. Apa perbedaan antara pengujian fungsional dan pengujian regresi?

Jawaban: Pengujian fungsional berfokus pada pengujian unit fitur baru secara spesifik untuk memastikan apakah fungsi atau metode tersebut telah memenuhi spesifikasi kebutuhan (misal menguji apakah fitur simpan dapat menulis file). Sedangkan **pengujian regresi** dijalankan terhadap seluruh suite pengujian sistem setelah terjadi perbaikan bug atau perubahan arsitektur (refactoring) guna memastikan bahwa modifikasi kode baru tersebut tidak secara tidak sengaja merusak atau mengubah perilaku fitur-fitur lama yang sebelumnya telah berjalan stabil.

2. Bagaimana refactoring dapat meningkatkan keterbacaan kode tanpa mengubah perilakunya?

Jawaban: Refactoring memperbaiki struktur internal perangkat lunak melalui teknik seperti ekstraksi metode (*Extract Method*), penamaan variabel dan metode yang lebih ekspresif (*Self-Documenting Code*), penghapusan duplikasi kode (*DRY Principle*), serta dekomposisi logika bersarang. Hal ini membuat alur kode menjadi jauh lebih mudah dipahami oleh pengembang lain tanpa mengubah kontrak antarmuka publik, parameter masukan, maupun nilai kembalian (*observable external behavior*) yang dirasakan oleh pemanggil.

3. Mengapa invarian kelas penting diidentifikasi sebelum pengujian dilakukan?

Jawaban: Invarian kelas menetapkan batas-batas mutlak dari status objek yang valid (misal: "kode tidak boleh kosong", "kapasitas tidak boleh bernilai negatif", "elemen dalam list tidak boleh memiliki kode ganda"). Mengidentifikasi invarian sebelum merancang test suite memungkinkan perekayasa perangkat lunak menentukan skenario batas (*boundary value analysis*) dan kondisi ekstrem (*edge cases*) yang paling berpeluang merusak objek, sehingga sistem dapat dibangun secara defensif untuk selalu mempertahankan status yang sah.

4. Apa risiko jika perbaikan bug dilakukan tanpa bukti dari langkah reproduksi yang jelas?

Jawaban: Melakukan perbaikan tanpa langkah reproduksi yang terbukti (reproducible steps) adalah tindakan spekulatif (*guessing-game*). Risikonya adalah pengembang hanya menutupi gejala luar (*symptom patching*) tanpa menuntaskan akar penyebab masalah yang sebenarnya (*root cause*). Selain itu, tanpa langkah pengujian reproduksi yang konsisten, tidak ada bukti empiris yang menjamin bahwa perbaikan tersebut benar-benar berhasil, bahkan berpotensi besar memicu bug baru (regresi) pada bagian lain.